

18 – More Python Class Methods

Clases en Python: Más Allá de la Estructura Básica

Construyendo Clases: El Diseño desde Adentro

En Python, una clase no es simplemente una agrupación de datos y funciones: es una forma de modelar el mundo. Define **qué es** un objeto (sus atributos) y **cómo se comporta** (sus métodos).

Tomemos como ejemplo la clase `Coordinate`. Esta clase representa un punto en el plano con coordenadas `x` e `y`, y ofrece métodos como `distance()` para calcular la distancia euclidiana entre dos puntos, o `to_origin()` para trasladar un punto al origen.

```
1 class Coordinate(object):
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance(self, other):
7         return ((self.x - other.x)**2 + (self.y - other.y)**2)**0.5
8
9     def to_origin(self):
10        self.x = 0
11        self.y = 0
```

Instanciación: Dar Vida a los Objetos

Crear una instancia significa materializar una clase: `c = Coordinate(3, 4)` genera un objeto concreto con valores únicos para sus atributos. A través del operador punto (`.`), accedemos tanto a sus datos como a sus comportamientos.

Composición de Clases: Clases Dentro de Clases

A menudo, clases se construyen utilizando otras clases. Un ejemplo elegante: la clase `Circle`, que tiene un `center` que es un objeto `Coordinate` y un `radius` como entero.

```
1 class Circle(object):
2     def __init__(self, center, radius):
3         if not isinstance(center, Coordinate) or not isinstance(radius,
4             int):
5             raise ValueError("Invalid types")
6         self.center = center
7         self.radius = radius
8
9     def is_inside(self, point):
10        return point.distance(self.center) < self.radius
```

Así, la clase `Circle` depende y colabora con `Coordinate`, ejemplificando la composición: una técnica poderosa de diseño orientado a objetos.

Operadores Especiales: Dunder Methods

Las operaciones cotidianas como `+`, `*` o `print()` se pueden redefinir dentro de nuestras clases usando **dunder methods** (double underscore methods). Son métodos mágicos que permiten que nuestras clases se comporten como tipos integrados de Python.

```
1 class Fraction(object):
2     def __init__(self, num, denom):
3         self.num = num
4         self.denom = denom
5
6     def __add__(self, other):
7         top = self.num * other.denom + self.denom * other.num
8         bottom = self.denom * other.denom
9         return Fraction(top, bottom)
10
11    def __str__(self):
12        if self.denom == 1:
13            return str(self.num)
14        return f"{self.num}/{self.denom}"
```

Con esto, podemos hacer:

```
1 f1 = Fraction(1, 4)
2 f2 = Fraction(1, 2)
3 print(f1 + f2) # Imprime: 6/8
```

Métodos Internos: Invertir y Reducir Fracciones

La clase `Fraction` también puede tener métodos como `invert()` (intercambia numerador y denominador) o `reduce()` (simplifica la fracción usando el máximo común divisor). Estos métodos son ejemplos de cómo encapsular lógica interna en comportamientos bien definidos.

```
1 def reduce(self):
2     def gcd(a, b):
3         while b != 0:
4             a, b = b, a % b
5         return a
6
7     g = gcd(self.num, self.denom)
8
9     return Fraction(self.num // g, self.denom // g)
```

Abstracción y Reutilización

La programación orientada a objetos permite que las clases oculten los detalles de implementación tras interfaces limpias. Cuando escribimos `f1 + f2` no necesitamos saber cómo está hecha la suma, solo confiamos en que “funciona”.

Esto nos da:

- **Modularidad**: el código está organizado.
- **Reusabilidad**: los objetos pueden usarse y extenderse fácilmente.
- **Mantenibilidad**: cambios internos no afectan a quien usa la clase.

Pensar como un Diseñador de Tipos

Aprender a usar clases en Python no es solo una cuestión de sintaxis. Es aprender a pensar en términos de **tipos**, **comportamientos** y **responsabilidades**. Es construir mundos a través del código.

“En cada objeto late un corazón lógico, y en cada clase, una arquitectura secreta del pensamiento.”

Ejercicio manual de la lectura

Implemente la clase que cumpla las siguientes especificaciones:

```
1  class Circle():
2      def __init__(self, radius):
3          """ Initializes self with radius """
4          self._radius = radius
5
6      def get_radius(self):
7          """ Returns the radius of self """
8          return self._radius
9
10     def __add__(self, c):
11         """ c is a Circle object
12         Returns a new Circle object whose radius is
13         the sum of self and c's radius """
14         return Circle(self._radius + c.get_radius())
15
16     def __str__(self):
17         """ A Circle's string representation is the radius """
18         return str(self._radius)
```